Developer Guide, Plain English Version

Rohith K Nair, DL.AI.U4AID25043, Developer

This explains what the developer actually built on this project, written for somebody with no background in programming or machine learning. There is a separate technical version of this guide with the code in it. This one has none.

If you read only one section, read section 7. It is the part that shows the work was done carefully rather than quickly.

---

1. What the developer's job was

Four of us worked on this project with different jobs. Mine was to write the code.

In practice that meant three things. I had to create the information the system learns from, because we could not obtain it any other way. I had to clean that information up and put it into a form a computer can work with. And I had to split it into two separate piles, one for teaching the system and one for testing whether it actually learned anything.

Everything in this guide sits in that first half of the project. Actually building the learning part comes next term.

---

2. The problem we are solving

When a company hires somebody new, a pile of paperwork follows. The new employee sends in a copy of their passport, a signed offer letter, their bank details, and depending on the job things like a safety certificate. Somebody in human resources has to look at each one and decide whether it is acceptable.

Meanwhile the new person has questions. How much holiday do I get. When is my training. Nobody has time to answer them quickly, so they wait.

Our system is meant to help with three parts of this. It reads the uploaded documents and decides whether they are valid. It answers questions from the company's own rulebook. And it notices when somebody is falling behind before a human does.

---

3. Why we had to invent our own information

This is the first thing people ask about, so it is worth explaining properly.

A system like this learns by looking at thousands of examples. To teach it what a passport looks like, you show it a few thousand passports. To teach it who struggles during their first month, you show it the records of a few thousand people who went through it.

We could not get either.

Real onboarding documents are people's passports and bank details. We cannot legally collect those from anybody, and there is no public collection of Indian employment paperwork we could borrow. As for the records of who struggled, those do not exist for us at all, because our system has never been used by a real company. There is nothing to learn from yet.

So we wrote programs that produce realistic fake documents, fake messages and fake employee histories. I wrote three of these, one for each kind of information we needed.

4. Making fake information that is not too easy

This sounds like it should be the simple part. It was not, and it is where I learned the most.

The obvious way to make fake documents is to write a few templates and fill in different names and dates. I did that first. Then we trained the system on it, and it got every single answer right.

That sounds like success. It is not. If a test is so easy that everybody scores full marks, the test tells you nothing about who actually understands the material. The same is true here. Our fake documents were too different from each other, so telling them apart required no real skill.

So I rebuilt the generator to make the problem genuinely hard. Three things changed.

Documents now come in three qualities. About four in ten are clean, as if scanned properly. Four in ten are damaged, with the top of the page missing and some letters scrambled. Two in ten are badly damaged, cropped down to a fragment, heavily scrambled, and with sentences from a completely different document accidentally mixed in.

The scrambling copies real mistakes. When a scanner misreads text it makes particular errors, confusing a zero with a capital O, or the letters r and n with an m. The scrambling uses those same confusions rather than random noise.

Some messages are deliberately impossible. The fake chat messages include ones like "any update?" and "still waiting". A person sending that had something specific in mind, but the words do not say what. I kept those in, labelled with what the person actually wanted. No system can get those right, which is the point. Real messages are like that too.

After these changes the hardest category scores about 94 out of 100 rather than 100. That is a number we can believe.

5. Turning words into numbers

Computers cannot work with words directly. They work with numbers. So a large part of my job was converting one into the other.

The method we use counts how often each word appears in a document, then adjusts that count based on how rare the word is overall. A word like "the" appears in everything, so it tells you nothing and gets a low weight. A word like "passport" appears in only one kind of document, so it gets a high weight. Every document becomes a long list of these weights.

There is a second trick that matters for our damaged documents. Alongside counting whole words, we also count short runs of three to five letters. The reason is that when a scanner scrambles a word, the whole word becomes unrecognisable, but most of the short letter runs inside it survive. So when the word-based method fails on a damaged document, the letter-based one still works. This is why the badly damaged category scores 94 instead of collapsing entirely.

For the numerical information about employees, the problem is different. One column holds a percentage from 0 to 100, another holds a number of days, another holds a score between minus one and plus one. Left alone, the column with the biggest numbers would dominate simply because its numbers are bigger. So every column is rescaled onto the same footing first.

6. Splitting the information in two

This is a small step that matters enormously, and it is one of the things the mid term review specifically asks about.

Before teaching the system anything, we divide our information into two piles. Three quarters goes into the teaching pile. One quarter is locked away and not touched.

Later, when we want to know whether the system actually learned anything, we test it on the locked-away quarter. Those are examples it has never seen.

If we skipped this and tested on the same examples we taught with, the system could simply memorise the answers and appear perfect while having understood nothing. It would be like setting an exam using the exact questions you gave out as practice.

There is a subtler version of the same mistake, where information leaks from the locked pile into the teaching process without anybody noticing. Guarding against that is section 7.

---

7. Three times the information surprised us

This is the section I would point a visitor to.

The first model was too good

Covered above in section 4. The model scored full marks, we treated that as a fault rather than an achievement, and rebuilt the data until it scored something believable.

What matters here is the instinct. It would have been easy to screenshot the perfect score and move on. Most people would.

We were wrong about our own data, and said so

While checking the split, I found that some of the fake chat messages appeared in both piles. Short messages like "thank you" get produced identically by chance, so the same text ended up in both the teaching pile and the locked-away pile.

That is a genuine mistake. My assumption was that it was making our results look better than they were, because the system was partly being tested on things it had already memorised.

Rather than assume, we measured it. We trained the system once, then scored the locked pile in two parts, the repeated messages and the fresh ones.

The result was the opposite of what I expected. The system did slightly **worse** on the repeated messages.

The reason, once we worked it out, makes sense. The messages that get repeated are the short vague ones like "thank you" and "any update?". Those are also the hardest to interpret. So the repetition is concentrated in the difficult cases, which drags that score down rather than lifting it.

The problem is real and still needs fixing. But our explanation for it was wrong, and we wrote that down rather than quietly deleting the evidence.

Three of our measurements were secretly the same measurement

We track twelve pieces of information about each employee. When I checked how they relate to each other, three of them turned out to move almost in lockstep: the stage somebody has reached, their percentage progress, and the fraction of their tasks finished.

They are three different names for roughly the same thing. This explained something that had been confusing us, which is why removing any one of them made almost no difference. Each looked useful on its own, but between them they carried only one piece of information.

---

8. What happens when somebody uploads a document

Walking through this end to end is the clearest way to see what the code does.

1. Somebody drags a file onto the website.
2. The system checks the file is really what it claims to be. It ignores the label the browser

puts on it and looks at the actual contents, because a file can be renamed to say anything.

1. It pulls the text out. Most documents are proper digital files with text already inside, so

this is quick. Only photographs and scans need the slower image reading step.

1. It decides what kind of document this is. Passport, safety certificate, bank form, or one

of nine possibilities.

1. It finds the important details. Dates, names, certificate numbers, and whether there is a

signature.

1. It applies the rules. Is this the document that was actually asked for. Is it still in

date. Does the name match. Is it signed where it needs to be.

1. It decides. Clear problem means reject with a reason. Everything clean and confident means

approve. Anything in between goes to a person.

That last step is deliberately cautious. Approving a bad document by mistake creates a real problem for the company. Sending a perfectly good document to a human by mistake costs somebody thirty seconds. The two errors are not equally bad, so the system is tuned to prefer the cheap mistake.

---

9. A mistake worth explaining

One fault we found is worth describing because of how it hid.

The system that predicts who is falling behind looks at twelve things about each employee. Two of those twelve were not actually being measured. The code was filling them in with the same fixed number for every single person, every time.

The system had been taught that those two things vary between people and matter. Then in practice it was handed a constant. It was applying what it had learned to information it was never really receiving.

Nothing crashed. No test failed. Every individual piece of the code worked correctly. It was only found by reading through the prediction path line by line and asking what each value actually was.

We checked whether either could be worked out from the information we hold. Neither could. So rather than invent them, we removed both and retrained. The score barely changed, which is exactly the point. It now describes what the system genuinely sees.

---

10. What I would do differently

Use real documents. Everything we report is bounded by the quality of the fake documents I wrote. There is a free public collection of four hundred thousand real scanned documents used by researchers, and training on that would turn our results from "consistent with our own invention" into "measured against reality". This is the single biggest improvement available.

Fix the duplicate messages properly. We measured the problem and found it small, but it is still a flaw and it should be removed rather than documented.

Merge the three overlapping measurements. Keeping three names for one thing makes it harder to explain which factors actually matter.

Put the work under version control from day one. We did not do this, and it means there is no history of changes and the project exists on one laptop. It is the least technical item on this list and probably the most important.

---

11. Summary

The developer work in this half of the project produced three sets of information totalling about fourteen thousand examples, a process for cleaning them and converting them into numbers, a split into teaching and testing piles that has been checked rather than assumed, and ten charts with written explanations of what each one shows.

The part I would defend most strongly is not any of those. It is that on three separate occasions the data told us something we did not want to hear, and each time we wrote it down instead of looking away.